Benchmarking Quality Diversity Methods for Instance Generation in Combinatorial Domains – Supplementary Material

Alejandro Marrero[1], Eduardo Segredo[1], Coromoto León[1] and Emma Hart[2]

[1] Departamento de Ingeniería Informática y de Sistemas, Universidad de La Laguna, San Cristóbal de La Laguna, Tenerife, Spain
{amarrerd, esegredo, cleon}@ull.edu.es
[2] School of Computing, Edinburgh Napier University, UK
e.hart@napier.ac.uk

A Parameter Setting

Parameterisations of the different approaches considered in this work are shown in Table 1.

Table 1: Parameter setting for the experiment evaluation of the QD methods

Parameter	**Value**
Dimension (N)	50 (KP and TSP), 120 (BP)
Population Size (M)	128
Number of neighbours (k) for NS and DNS	15
Linear Weight coefficient (ϕ)	0.85
Number of repetitions per solver (R)	1
Generations	1000
Mutation	Uniform One
Crossover	Uniform
Crossover and mutation rates	0.5, 0.8
Thresholds for NS	
KP (t_a, t_s)	3 ($NS_{f,p,i}$)
TSP (t_a, t_s)	3 ($NS_{f,i}$), 1e-7, 1-e10 (NS_p)
BP (t_a, t_s)	1e-7
Regions and number of samples for CVT	1,000 and 100,000

B Threshold tuning through iRace

We run iRace [4] to tune the archive and solution set thresholds of NS_f method with the aim of maximising the value of the exploration uniformity metric U. For each domain, we run 512 trials (following iRace nomenclature) to find the t_a and t_s that maximise the uniform coverage of the reduced instances after PCA (U). The search range for each parameter and domain is as follows: $t_{a,s} \in [0.0, 10.0]$ for KP and TSP and $t_{a,s} \in [1e-8, 1.0]$ for BP. Results (Table 2) show that the tuned version (NS_f with t_a and t_s found by iRace) scores a higher U for each domain evaluated. However, it is important to highlight that the tuned version produced larger sets of instances, which could potentially lead to better coverage of the space. The threshold values found for each domain are: KP $t_a = 7.0950$ and $t_s = 3.5748$, TSP $t_a = 0.0991$, and $t_s = 5.2792$, and BP $t_a = 0.5753$ and $t_s = 0.0202$.

Table 2: Summary of the results of best configurations found by iRace, its counterpart with the parameter settings found in literature for each domain. Values under target solvers for each domain represent the unique instances generated by each configuration.

Method	**KP**					**BP**					**TSP**			
	U	**Def**	**MaP**	**MiW**	**MPW**	U	**BF**	**FF**	**NF**	**WF**	U	**2-Opt**	**Greedy**	**NNeighbour**
NS_f	0.5116	381,512	553,063	269,838	615,129	0.5660	1,261,599	1,168,136	16,362	976,134	0.5116	131,819	131,841	133,160
NS_{ft}	0.5534	772,130	925,310	617,070	1,267,045	0.5633	465,086	617,744	8,539	558,698	0.5534	0	1,273,655	1,274,300

C Meta-Quality Diversity (Meta-QD)

Meta-QD is an adaptation of a QD algorithm in which an evolutionary algorithm is used to learn a non-linear 2D projection of an original feature-space such that applying a QD method in the 2D space to generate instances improves the coverage of the instance space. This avoids having to pre-define a descriptor. The idea was first proposed in the context of QD in [1, 2] to learn the axes of a grid used in MAP-Elites for a robotics application; we adapted this method to work with novelty search in [5]. Specifically, the meta algorithm consists of an outer loop that evolves a population in which each individual provides the weights of a fixed architecture neural network — evolution of this loop is achieved via the well-established method CMA-ES [3]. Each network in this population provides a mapping from a high-dimensional feature vector to 2D.[3] The fitness of each individual in the outer population is obtained by running a complete NS algorithm, where the descriptors are obtained from the neural network specified by the outer individual. The coverage of the instance space by the generated instances obtained when the NS algorithm completes is assigned as the objective value of the individual in the outer loop, which drives the evolutionary process. We refer to this method as *meta-QD*.

This *meta-QD* approach cannot be directly compared to the other methods, as it requires a substantial pre-learning phase to determine the mapping. Specifically, the meta-algorithm runs CMA-ES for 4000 evaluations (16 population size $\times$ 250 generations), and for each of these evaluations, a full NS algorithm is executed for 10,000 evaluations. This results in a total of 40 million evaluations solely for learning the projection, which is not feasible to include in standard benchmarking comparisons. The method was run ten times, and then we selected the best model for each domain.

Then, the best model found was combined with DNS and NS to generate instances for each domain addressed. The results can be found in Table 3. Results show that the 'meta' descriptor approach obtained the highest U score in the KP domain, and competitive scores in the TSP domain. Additionally, DNS outperformed NS in each domain.

[3] The meta algorithm could use any method to learn the mapping from high-d to low-d, e.g. it could learn the weights of an Autoencoder or the parameterisation of a dimensionality reduction method such as UMAP.

Table 3: Statistics of the KP, BP and TSP instances generated by each meta-QD configuration. The value under each solver column specifies the number of unique instances generated by that QD configuration for such target. Total represents the number of instances generated after ten repetitions of each run. Unique instances are calculated based on the BP instance definition, regardless of the method and descriptor used to generate them.

Domain	**Method**	U	**Total**	**Ratio Unique**	**Portfolio**			
					Def	*MaP*	*MiW*	*MPW*
KP	DNS_{meta}	**0.5678**	5,120	0.9988	1,280	1,278	1,276	1,280
	NS_{meta}	0.5669	3,861,633	0.9933	871,521	1,039,112	672,402	1,253,629
					BF	*FF*	*NF*	*WF*
BP	DNS_{meta}	**0.4552**	2,688	0.9349	1,215	1,176	0	122
	NS_{meta}	0.4195	1,276,880	0.9895	1,174,167	89,145	0	124
					2-Opt	*Greedy*	*NNeighbour*	
TSP	DNS_{meta}	**0.5703**	3,840	0.9960	1,280	1,273	1,272	
	NS_{meta}	0.5540	1,287,884	0.9994	10	1,044,890	242,344	

While we acknowledge that such methods can offer improvements in instance coverage when sufficient computational resources are available, they are not directly comparable with methods that do not require a pre-learning phase. As such, we consider the inclusion of the meta-algorithm results as a qualitative indicator of potential performance gains, rather than a direct benchmark. In future work, we may explore more efficient meta-learning strategies or separate the computational cost of pre-training from the evaluation phase to enable fairer comparisons.

References

1. Bossek, J., Neumann, F.: Exploring the Feature Space of TSP Instances Using Quality Diversity. In: Proceedings of the Genetic and Evolutionary Computation Conference. p. 186–194. GECCO '22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3512290.3528851, https://doi.org/10.1145/3512290.3528851
2. Bossens, D.M., Mouret, J.B., Tarapore, D.: Learning behaviour-performance maps with meta-evolution. In: Proceedings of the 2020 Genetic and Evolutionary Computation Conference. pp. 49–57 (2020)
3. Hansen, N.: The cma evolution strategy: A tutorial (2023), https://arxiv.org/abs/1604.00772
4. López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L.P., Birattari, M., Stützle, T.: The irace package: Iterated racing for automatic algorithm configuration. Operations Research Perspectives **3**, 43–58 (2016)
5. Marrero, A., Segredo, E., León, C., Hart, E.: Learning descriptors for novelty-search based instance generation via meta-evolution. In: Proceedings of the Genetic and Evolutionary Computation Conference. p. 206–213. GECCO '24, Association for

Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3638529.3654028, https://doi.org/10.1145/3638529.3654028